SHOP NOTES

THE MANUAL AS TRAINING DATA

why the one part of this repository that the program writes about itself is the part worth teaching to a small model, what the five steps are, and which pieces of them exist today

---

Notes on work in progress · issue #192

James Andrew Walsh

August 2026

In one line: Organon writes its own reference manual out of its source code, and a test fails the build if that manual and the code ever disagree, which makes it an unusually clean thing to train a small language model on, so this note is the plan for doing that, the parts that already exist, and the four ways it could fail.

JAMES' WORKSHOP

make the thing · show the thing · write it down plainly

Why this note exists

I want to try something that is easy to describe badly, so I am writing it down before any of it is built.

Organon is a program for making moving three dimensional forms. It has a catalog: a list of the shapes it can generate, the ways those shapes can be turned into surfaces, the materials those surfaces can be made of, and every setting you can change. That catalog is written down in the manual, and the manual is not typed by hand. The program writes it, out of its own source code, and a test fails if the two ever drift apart.

The idea in this issue is to take that manual and use it as teaching material for a small language model, then measure whether the teaching worked, and eventually have that model operate the program on this machine with no network connection at all. What follows is what that would mean, which pieces of it exist in the repository today (a few, and not the ones you might guess), which do not exist at all (most), and the places where I think it can fail.

None of the five steps below is built. Two of the harder supporting pieces are, with tests. Nothing has been trained. Every number I quote about the eventual result is a projection, and I say so each time.

The part of the repository that cannot quietly go stale

Counted this morning, the catalog has **27 generators, 10 surfaces, 8 materials, 48 settable parameters and 7 recipes**: a hundred entries. A recipe is a named starting point, something like "Refracting DNA" or "Nebula Fire", that sets a couple of dozen controls at once.

Every one of those descriptions lives in the Rust source, in the same file as the code it describes. A command, `organon docs`, reads them and writes out the manual pages under `doc/reference/`. Those pages are committed. A test regenerates them and compares, and fails the build if a single byte differs. So the description of a setting and the setting itself cannot get out of step: changing one without the other stops the build.

That gives this particular body of text five properties that scraped training data does not have.

- **It is committed.** Anyone who clones the repository gets the same bytes I have.
- **It is regenerable.** A checkout at any commit produces that commit's version of the manual, not today's.
- **It cannot rot in silence.** The test is the guarantee, and it already exists for its own reasons.
- **A general model cannot already know it.** The recipe names, the parameter ranges and most of the vocabulary were invented here. Ask a model that has never seen this repository what Organon's "Nebula Fire" recipe sets and it will produce a confident paragraph of fiction.
- **It is already shaped like teaching material.** Every entry is an answer waiting for its question.

The fourth is the one that makes the whole exercise measurable. Because the base model demonstrably does not know this vocabulary, checking whether the training took is a single question typed at a prompt, not an evaluation suite I would have to build and then trust.

Measured today with a word count: the reference pages are **5,337 words** and the hand written guide beside them is **5,228**, so about **10,600 words** in total. Hold on to that number, because it comes back in the last section as the most likely reason this does not work.

Why not train on everything

The tempting answer is to feed in the whole repository: the architecture notes, the running history of changes, the reasoning that goes with each one. It is richer material and it carries the voice of the project rather than just its vocabulary.

The problem is churn. The point of the exercise is to see a difference: change a description, retrain, and observe that the model changed in a way you can attribute to what you wrote. A body of text that changes twice a week smears that difference into noise.

So the scope was chosen by counting rather than by taste. The issue counts six months of the project's history from before this repository was public: the reference pages were touched by **2 commits out of 527**, the guide by 3, the architecture notes by 141, and the running notes on each change by 155.

This repository has a log of its own, 542 commits since it opened on the ninth of August, and counting again there gives the same answer: **reference twice, guide three times, the main architecture document 27 times, the second one 100 times, and the notes on each change 168 times.** The two totals landing within fifteen commits of each other is a coincidence rather than a sign they are the same log, since one covers half a year and the other a fortnight.

That is a spread of nearly two orders of magnitude between the calmest text in the repository and the busiest, and both counts agree about which end is which. Start with the calm end.

The five steps

One: the corpus. A sibling of `organon docs` that emits question and answer pairs from the same Rust definitions the manual comes from, committed and pinned by a test exactly the way the manual is. This step is worth having even if I never train anything: it is a reviewable, diffable file showing precisely what a model would be taught. It needs no graphics card and no training software.

Two: the recipe. Which base model, which random seed, how large the adjustment is allowed to be, which parts of the network it is applied to, how many steps. As a file in the repository, not a sequence of clicks in an application, so that "anyone who clones this can reproduce it" is a claim someone can check rather than one I get to assert.

Three: the fingerprint. After a training run, two numbers for each place in the network that was adjusted: how far the weights moved, and how concentrated the movement was. For a 48 layer model with 7 adapted parts per layer that is 336 places, which is under 50 kilobytes of text. **The measurement gets committed, never the model.** The trained adjustment is about 40 megabytes and the base model it attaches to already lives on a public host, so committing either would be storing someone else's file badly.

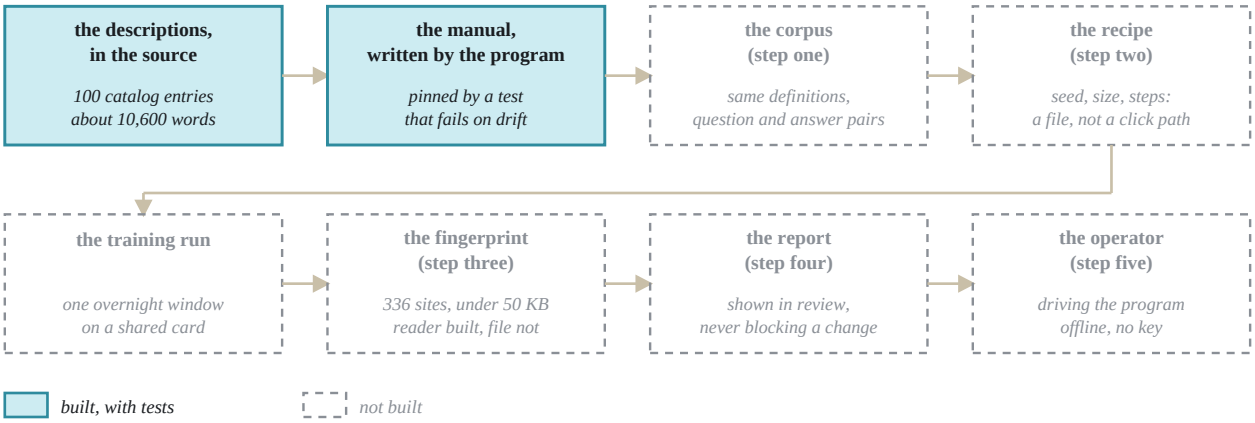
Four: the report. When a change to a description comes up for review, show what it did to the model: this rewrite moved the fourteenth layer by twelve percent. The experiment as a thing that happens on its own rather than something I have to remember to run.

Five: the operator. The trained model driving Organon through its own command line, on the machine under the desk, with no network connection and no interface key. The vocabulary living in the weights instead of

being pasted into a prompt every time. This is the door the whole issue exists to open, and it is the step I am least sure about.

What exists today

From a sentence in the source to a model that can drive the program



Committed to the repository: the manual, the corpus, the recipe, the measurement. Not committed: the trained adjustment of about 40 megabytes, and the base model it attaches to, which is published elsewhere.

The chain from a sentence in the source code to a model that can operate the program. The two shaded boxes are built and have tests: the program writes its own manual and a test fails if the manual and the code disagree, and the reader that measures what a training run moved is finished, with 40 tests, along with 30 more for the connection to the training application. Everything drawn with a broken outline is not built, which is five of the six remaining boxes and all five of the steps in the issue. The line underneath is the storage decision: the corpus, the recipe and the measurement are small, readable and committed, while the trained adjustment is about 40 megabytes and the base model belongs to somebody else, so neither of those is kept here.

The manual and the test that pins it are real, and have been for a while. They were built for their own sake, which is why this idea is cheap to try.

Two pieces further down the chain are also already here, and they are the expensive ones.

The reader for a trained adjustment exists, with 40 tests. Give it the directory a training run produces and it reports, for every adapted site, exactly the two numbers step three wants. It does that without ever building the full weight change, which matters: the change at one site is 4096 by 4096 numbers and there are hundreds of sites, but it is stored as a product of two thin matrices, so both quantities can be computed through a small square in the middle instead. For a site of that size that is between about thirty and two hundred and fifty times less arithmetic than the direct route, depending on how large the adjustment is allowed to be, which is the difference between a readout and a wait. There is also code that folds those numbers into positions on the drawn model, so the measurement has somewhere to go.

The connection to the local training application exists, with 30 tests. It answers one question, whether we can talk to the training service, and it distinguishes three ways the answer can be no: not configured, not running, and not authorised. Those are three different problems with three different fixes, and collapsing them into "cannot connect" sends a person to restart an application when what they actually need is a key they never created.

Now the part that is not built, which is most of it. **There is nothing in the tree that emits training pairs.** The `docs` command writes markdown and stops. There is no recipe file, no fingerprint file, no report, and no operator. No training run has happened, so the 336 site figure above is arithmetic rather than an observation, and the question of whether any of this teaches the model anything worth having stays completely open until the first run happens.

Three rules I would rather not re-derive later

This is not a build step. The obvious shape, retrain whenever the documentation changes, sounds tidy and is wrong by two orders of magnitude: on the six month history above it would have fired several hundred times to catch a handful of real changes. The right comparison is a dependency lockfile, which regenerates when its declared input changes, gets committed, and gets read in review. Three costs, three homes: regenerating the corpus takes seconds and belongs in the build; retraining takes minutes to hours and belongs on the corpus changing; the fingerprint is committed with the retrain. And because the corpus is committed and readable, I can judge whether a change is worth the electricity before spending it. A typo in a description, no. A new generator, yes.

Report, never block. A pull request refused because a number moved is a check that everyone learns to click past, which is worse than not having the check at all.

Keep a person in the loop. If the model is trained on the documentation, and the documentation is then judged by whether the model learned it, the whole arrangement can converge on something perfectly self consistent and wrong. The instrument takes the measurement, the measurement carries a label saying what kind of number it is, and a person decides what to change. That is the difference between a feedback loop and a mirror.

There is also one placement decision worth stating. **The corpus builder goes in the program, beside the command that writes the manual, and not in a script that reads the manual.** Anything parsing the generated markdown is downstream of it and can drift away from the source; the whole value here comes from both artifacts being generated from the same definitions.

What could make this fail

10,600 words is probably thin. That is a short pamphlet, and I am proposing to move a model of billions of parameters with it. The obvious fix is to expand each entry into several phrasings of the same fact, but that expansion has to be generated and committed too, or the reproducibility claim quietly stops being true. Whether the result carries enough signal to shift anything measurably is unknown until step three runs once.

Knowing the words is not knowing the job. Training on documentation teaches a model to reproduce documentation. A model that can recite every parameter and its range can still make consistently poor choices about which one to reach for, and step five is exactly where that gap would appear. If the first four steps go well, that is not evidence about the fifth.

Training is not repeatable to the last digit. The seed is settable, but the arithmetic on a graphics card is not run in a fixed order, so two runs on the same machine differ slightly and two different machines differ more. The fingerprint therefore needs a stated tolerance, and its real scope is: tightly comparable between commits on

one machine, loosely comparable across machines. If I do not write that down, someone reasonably files a bug about the fifth decimal place.

The graphics card is already busy. The machine that would do the training also runs a speech model that reclaims most of the card's memory whenever it is asked to talk, so a retrain wants an overnight window. And a retrain on a schedule, running whether or not anything changed, would produce a couple of hundred nearly identical fingerprints a year, each one a commit, with the single one that mattered invisible among them. The trigger is the corpus changing, not the calendar.

The model for step five is unchosen. There is a reference specimen in the issue, but a model that operates the program locally, next to the speech stack, on one consumer card, may want to be far smaller than the one used to test whether the documentation teaches anything at all. That is a separate measurement and I do not want it assumed.

What I would like an opinion on

Two things, before I build any of it.

First, the scope. I have argued for the calm 10,600 words on the grounds that a stable corpus makes the difference visible. The counterargument is that it is simply too small to move anything, and that the right move is the broad set with worse resolution. I would rather hear that before the first training run than after it.

Second, the unit that gets committed. I think 50 kilobytes of per site measurements is the right thing to keep in the repository forever, and that the trained adjustment itself is not. If you have watched a project try to version model weights and regret it, or not regret it, I would like to know which.

The first step, the corpus itself, is worth building regardless of how those two questions land, because a file showing exactly what a model would be taught is a thing I want to read whether or not anything is ever trained on it.